ЛР №1 · вариант 26 · база знаний системы NIKA

Предметная область языков программирования

23	17	40	70	77
АБСОЛЮТНЫХ ПОНЯТИЯ	ОТНОСИТЕЛЬНЫХ	ЭКЗЕМПЛЯРА	СВЯЗЕЙ	ФАЙЛОВ .SCS

ТРЕБОВАНИЯ МЕТОДИЧКИ И ЧТО ИМ СООТВЕТСТВУЕТ

ТРЕБОВАНИЕ	ЧТО ЕСТЬ	ЧЕМ ЗАДАНО
✓ Выбрать тему согласно варианту	Вариант 26 — «Языки программирования»	Тему нужно согласовать с преподавателем и убедиться, что в потоке она уникальна.
✓ Формализовать структуру предметной области	10 максимальных класса, 13 немаксимальных, 17 исследуемых отношений	Задано узлом `subject_domain_of_programming_languages` внутри контура раздела.
✓ Не менее 30 абсолютных и относительных понятий	40 понятий: 23 абсолютных + 17 относительных	Норма перекрыта на треть.
✓ Теоретико-множественные отношения между понятиями	Надкласс у каждого из 23 понятий, 5 разбиений	`nrel_inclusion` и `nrel_subdividing`.
✓ Свойства и домены у формализуемых отношений	У всех 17 отношений	Первый и второй домен, область определения и пять свойств: бинарное, ориентированное, антирефлексивное, асимметричное, антитранзитивное.
✓ Примечания, пояснения и определения	Все три вида у каждого из 23 понятий	`nrel_note`, `nrel_explanation` и структура `definition` с переводом на естественный язык.
✓ Тексты из открытых источников	Ссылка на Википедию у каждого понятия	Отношение `nrel_source`; существование всех 23 статей проверено.
✓ Не менее одного экземпляра на каждое понятие	40 экземпляров, понятий без экземпляра нет	Разложены по девяти папкам внутри `concepts/instances/`.

ТРЕБОВАНИЕ	ЧТО ЕСТЬ	ЧЕМ ЗАДАНО
✓ Экземпляры связаны относительными понятиями	70 связей, изолированных узлов нет	Каждое отношение задействовано минимум дважды.
• Протестировать фрагменты в системе NIKA	Нужен запуск на Linux	Сборка базы знаний идёт при каждом старте `docker compose up`.
• Составить электронный отчёт	После скриншотов из sc-web	Цель, задание, вариант, иллюстрации, тестирование, выводы.
• Уметь отвечать на вопросы и решать задачи	19 вопросов и 3 задачи в конце ЛР	Самая весомая часть аттестации — 3 балла из 10.
• Запушить изменения в форк на GitHub	После проверки сборкой	Папка целиком в `kb/extra/`.

> Закрыто 9 пунктов из 13. Оставшиеся четыре не делаются файлами: они требуют запущенной системы, скриншотов и подготовки к защите.

ЧТО ТАКОЕ АБСОЛЮТНОЕ И ОТНОСИТЕЛЬНОЕ ПОНЯТИЕ

Абсолютное понятие — это класс, множество объектов с общими свойствами. Отвечает на вопрос «что это за сущность». В файлах помечается `sc_node_class`, системный идентификатор начинается с `concept_`. Конкретный объект класса называется его экземпляром.

Относительное понятие — это отношение, множество связок между сущностями. Отвечает на вопрос «как сущности связаны». Делится на два вида, и различить их на защите спрашивают почти всегда:

`nrel_ · *` **Неролевое отношение.** Самостоятельная связь между двумя сущностями: «у языка есть автор». В русском идентификаторе на конце звёздочка.

`rrel_ · '` **Ролевое отношение.** Подмножество отношения принадлежности: указывает роль элемента внутри множества — «эта реализация среди прочих эталонная». В русском идентификаторе на конце апостроф.

Префиксы `nrel_` и `rrel_` нужны потому, что символы `*` и `'` запрещены в алфавите системных идентификаторов — там допустима только латиница.

Максимальный класс объектов исследования — тот, у которого в этой предметной области нет надмножества среди других исследуемых

классов. У нас таких 10. Немаксимальные — их подклассы, например компилируемый язык внутри языка программирования.

МАКСИМАЛЬНЫЕ КЛАССЫ ОБЪЕКТОВ ИССЛЕДОВАНИЯ · 10

ПОНЯТИЕ	СИСТЕМНЫЙ ИДЕНТИФИКАТОР	СВЯЗИ С ДРУГИМИ ПОНЯТИЯМИ
язык программирования	concept_programming_language	⊂ формальный язык 4 разбиения, 10 подклассов
парадигма программирования	concept_programming_paradigm	⊂ методология программирования
реализация языка программирования	concept_programming_language_implementation	1 разбиение, 3 подкласса
стандарт языка программирования	concept_language_standard	⊂ нормативный документ
разработчик языка программирования	concept_programming_language_developer	⊂ человек
организация-разработчик программного обеспечения	concept_software_development_organization	⊂ организация
стандартная библиотека	concept_standard_library	⊂ библиотека программных компонентов
интегрированная среда разработки	concept_integrated_development_environment	⊂ программное средство
менеджер пакетов	concept_package_manager	⊂ программное средство
календарный год	concept_calendar_year	⊂ промежуток времени

НЕМАКСИМАЛЬНЫЕ КЛАССЫ · 13

ПОНЯТИЕ	СИСТЕМНЫЙ ИДЕНТИФИКАТОР	СВЯЗИ С ДРУГИМИ ПОНЯТИЯМИ
компилируемый язык программирования	concept_compiled_programming_language	⊂ язык программирования
интерпретируемый язык программирования	concept_interpreted_programming_language	⊂ язык программирования
язык программирования высокого уровня	concept_high_level_programming_language	⊂ язык программирования

ПОНЯТИЕ	СИСТЕМНЫЙ ИДЕНТИФИКАТОР	СВЯЗИ С ДРУГИМИ ПОНЯТИЯМИ
язык программирования низкого уровня	concept_low_level_programming_language	⊂ язык программирования
объектно-ориентированный язык программирования	concept_object_oriented_programming_language	⊂ язык программирования
функциональный язык программирования	concept_functional_programming_language	⊂ язык программирования
процедурный язык программирования	concept_procedural_programming_language	⊂ язык программирования
логический язык программирования	concept_logic_programming_language	⊂ язык программирования
язык программирования со статической типизацией	concept_statically_typed_programming_language	⊂ язык программирования
язык программирования с динамической типизацией	concept_dynamically_typed_programming_language	⊂ язык программирования
компилятор	concept_compiler	⊂ реализация языка программирования
интерпретатор	concept_interpreter	⊂ реализация языка программирования
виртуальная машина	concept_virtual_machine	⊂ реализация языка программирования

НЕРОЛЕВЫЕ ОТНОШЕНИЯ · 14

ОТНОШЕНИЕ	СИСТЕМНЫЙ ИДЕНТИФИКАТОР	ПЕРВЫЙ ДОМЕН → ВТОРОЙ ДОМЕН
автор языка программирования*	nrel_language_author	язык программирования → разработчик языка программирования
год создания языка программирования*	nrel_creation_year	язык программирования → календарный год
источник сведений*	nrel_source	язык программирования → файл (sc-ссылка)
менеджер пакетов языка*	nrel_package_manager	язык программирования → менеджер пакетов

ОТНОШЕНИЕ	СИСТЕМНЫЙ ИДЕНТИФИКАТОР	ПЕРВЫЙ ДОМЕН → ВТОРОЙ ДОМЕН
место работы разработчика*	nrel_developer_organization	разработчик языка программирования → организация-разработчик программного обеспечения
поддерживаемая парадигма программирования*	nrel_supported_paradigm	язык программирования → парадигма программирования
поддерживается средой разработки*	nrel_ide_support	язык программирования → интегрированная среда разработки
пояснение*	nrel_explanation	язык программирования → файл (sc-ссылка)
реализация языка*	nrel_language_implementation	язык программирования → реализация языка программирования
создан под влиянием языка*	nrel_influenced_by	язык программирования → язык программирования
сопровождающая организация*	nrel_maintaining_organization	язык программирования → организация-разработчик программного обеспечения
стандарт языка программирования*	nrel_language_standard	язык программирования → стандарт языка программирования
стандартная библиотека языка*	nrel_standard_library	язык программирования → стандартная библиотека
язык реализации*	nrel_implementation_language	реализация языка программирования → язык программирования

РОЛЕВЫЕ ОТНОШЕНИЯ · 3

ОТНОШЕНИЕ	СИСТЕМНЫЙ ИДЕНТИФИКАТОР	ПЕРВЫЙ ДОМЕН → ВТОРОЙ ДОМЕН
действующая редакция стандарта'	rrel_current_standard	язык программирования → стандарт языка программирования
основная парадигма'	rrel_main_paradigm	язык программирования → парадигма программирования
эталонная реализация'	rrel_reference_implementation	язык программирования → реализация языка программирования

Справа у каждого экземпляра — число исходящих связей. Пустое поле означает, что экземпляр только принимает связи от других: например, Ален Колмероэ связан входящей дугой «автор языка программирования*» от Пролога.

Языки программирования 8

Python `python_language`	11
Си `c_language`	10
Си++ `cpp_language`	10
Java `java_language`	9
JavaScript `javascript_language`	8
Haskell `haskell_language`	5
Пролог `prolog_language`	4
язык ассемблера x86 `assembly_language_x86`	1

Парадигмы 4

функциональное программирование `paradigm_functional_programming`	
логическое программирование `paradigm_logic_programming`	
объектно-ориентированное программирование `paradigm_object_oriented_programming`	
процедурное программирование `paradigm_procedural_programming`	

Реализации языков 3

CPython `cpython_interpreter`	1
GCC `gcc_compiler`	1
виртуальная машина Java `java_virtual_machine`	1

Стандарты 3

ECMA-262 `ecma_262`	
ISO/IEC 14882:2020 `iso_iec_14882_2020`	
ISO/IEC 9899:2018 `iso_iec_9899_2018`	

Разработчики 7

Бьёрн Страуструп `bjarne_stroustrup` 1

Деннис Ритчи `dennis_ritchie` 1

Гвидо ван Россум `guido_van_rossum` 1

Джеймс Гослинг `james_gosling` 1

Ален Колмероэ `alain_colmerauer`

Брендан Эйх `brendan_eich`

Саймон Пейтон Джонс `simon_peyton_jones`

Организации 4

Bell Labs `bell_labs`

Международная организация по стандартизации `iso_organization`

Oracle Corporation `oracle_corporation`

Python Software Foundation `python_software_foundation`

Стандартные библиотеки 2

стандартная библиотека Си `c_standard_library`

стандартная библиотека Python `python_standard_library`

Инструменты 4

npm `npm_package_manager`

pip `pip_package_manager`

PyCharm `pycharm_ide`

Visual Studio Code `visual_studio_code_ide`

Календарные годы 5

1972 год `year_1972`

1985 год `year_1985`

1990 год `year_1990`

1991 год `year_1991`

1995 год `year_1995`

КАК ЧИТАТЬ ФАЙЛ .SCS

```
/* фрагмент concepts/concept_compiler.scs */
concept_compiler
<- sc_node_class;
<- concept_wit_entity;
=> nrel_main_idtf:
    [компилятор]
    (* <- lang_ru;; *);
<= nrel_inclusion:
    concept_programming_language_implementation;;
```

`<-`	принадлежность: компилятор — *элемент* класса `sc_node_class`
`=>`	связка *исходит* от компилятора к идентификатору
`<=`	связка *входит* в компилятор — значит реализация языка является надмножеством, а компилятор её подклассом
`[ ]`	sc-ссылка: текст на естественном языке
`(* *)`	контур: всё внутри относится к предыдущему элементу
`;`	перечисление продолжается, субъект тот же
`;;`	конец предложения
`...`	анонимный элемент без имени — так задаются определения и разбиения

> Направление дуги здесь несёт смысл. Перепутанные `<=` и `=>` у `nrel_inclusion` превратят надкласс в подкласс — база соберётся, но иерархия будет вывернута наизнанку.
